CYBERSECURITY AVANZATA · LEZIONE 5 · OAUTH, JWT E PASSKEY

# Un'API con il token

Unisci le unità: un web service REST in Java (unità 2) che rilascia e controlla JWT firmati (lezione 5), con le password conservate bene (lezione 3).

**Cosa consegni:** un file **api-token-cognome.zip** con il codice Java e un **README.pdf** con i comandi curl e le risposte, su Google Classroom. **Scadenza:** quella indicata su Novaula.

## Che cosa devi fare

1. Con `HttpServer` crea **POST /login**: riceve `{"utente":"…","password":"…"}`, verifica la password (PBKDF2 come nel compito precedente, oppure un utente di prova fisso) e risponde con un JWT **HS256** che contiene `sub`, `ruolo` e `exp` = adesso + 15 minuti.
2. Il segreto di firma: 32 byte da `SecureRandom`, generati all'avvio. Mai scritto nel codice.
3. Crea **GET /api/profilo**: legge l'header `Authorization: Bearer <token>`, verifica firma (con `MessageDigest.isEqual`) e scadenza, e risponde **200** con i dati del token, altrimenti **401**.
4. Crea **GET /api/admin**: risponde **403** se il ruolo non è `docente`.
5. Prova con curl: senza token, con token valido, con token manomesso (ruolo cambiato), con token scaduto (metti per prova `exp` nel passato). Incolla comandi e risposte nel README.

## Esempio

`curl -H "Authorization: Bearer eyJ…" http://localhost:8080/api/profilo` → `200 {"sub":"studente42","ruolo":"studente"}`
stesso token con il ruolo cambiato → `401` · `/api/admin` da studente → `403`

> *"Su Internet nessuno sa che sei un cane."* · **Peter Steiner**, *The New Yorker (1993). Con un token firmato, almeno l'API lo sa.*

## Come viene valutato

| Criterio | Punti |
|---|---|
| Login che rilascia un JWT corretto (tre parti, Base64url, HS256) | 3 |
| Verifica di firma e scadenza, 401 nei casi giusti | 3 |
| Controllo del ruolo con 403 | 1 |
| Quattro prove con curl documentate | 2 |
| Segreto generato bene, nessuna password in chiaro | 1 |

Nei progetti veri si usa una libreria JWT collaudata: scriverla a mano qui serve a capire che cosa controlla.